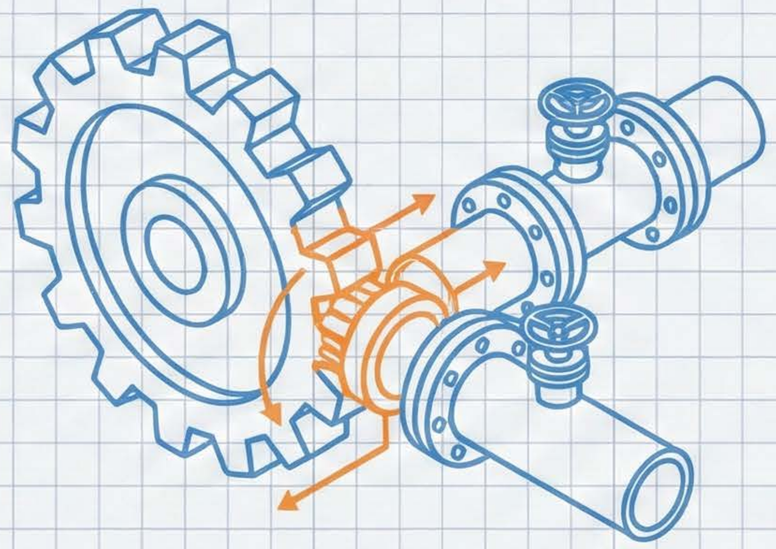
Unit 02 Python 程式語言基礎

Section 05-06：函式與模組

建構數位化工廠：從單元操作到系統整合



Python for Chemical Engineering & Data Science

NotebookLM

函式 (Functions)：程式碼的單元操作

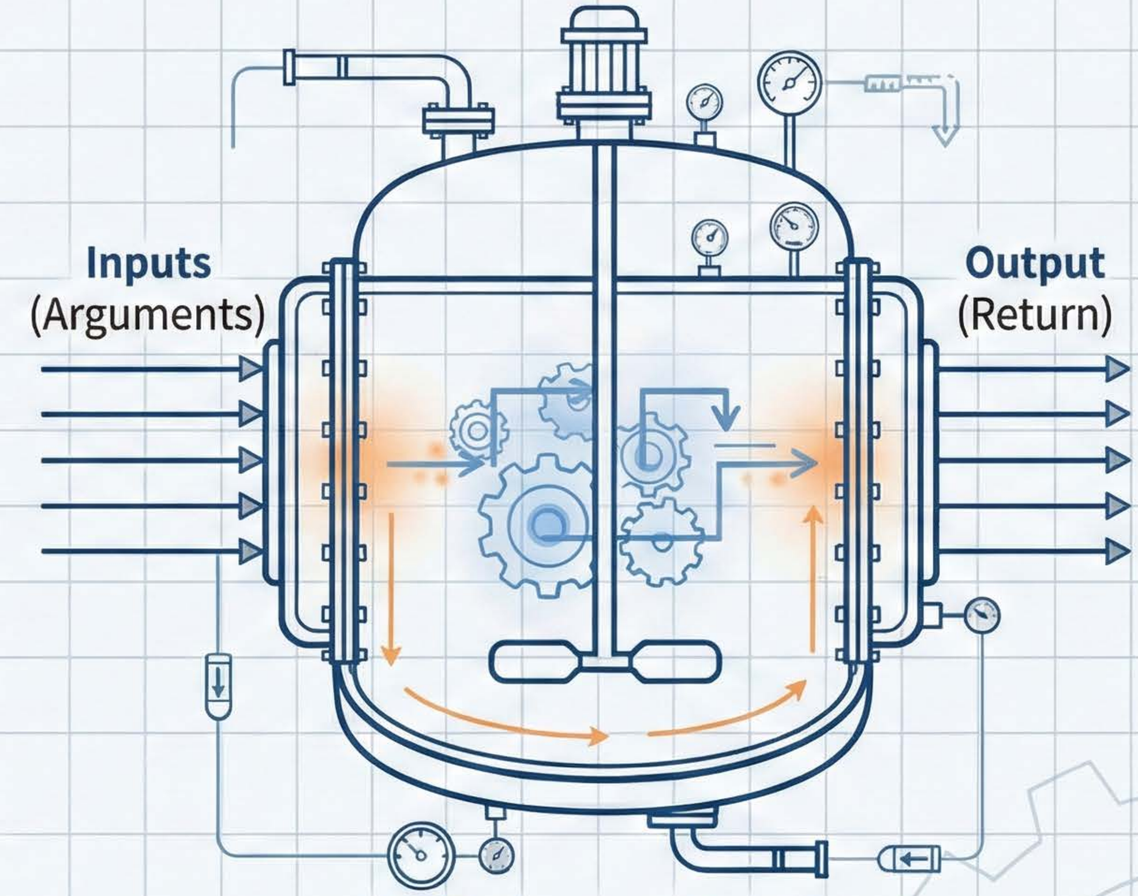
什麼是函式？

- **定義**：一段可重複使用的程式碼區塊，用於執行特定任務。
- **工程類比**：函式就像化工廠裡的反應器 (Reactor) 或單元操作 (Unit Operation)。
  - 不需要為每一批產品重新建造反應器。
  - 只需建造一次，即可重複投料生產。

Input (輸入)
原料 → 參數
(Arguments)

Process (處理)
反應 → 函式內部邏輯

Output (輸出)
產品 → 回傳值
(Return Value)



CSTR Reactor R-101

核心價值：DRY (Don’t Repeat Yourself) — 避免重複造輪子，提升維護效率。

語法解構：打造你的第一個反應器

宣告關鍵字
(Start of Specification)

函式名稱
(Equipment Tag)

參數
(Inputs/Feeds)

```
def calculate_density(mass, volume):
    """
    計算密度的函式 (Docstring)
    """
    if volume == 0:
        return None

    density = mass / volume
    return density
```

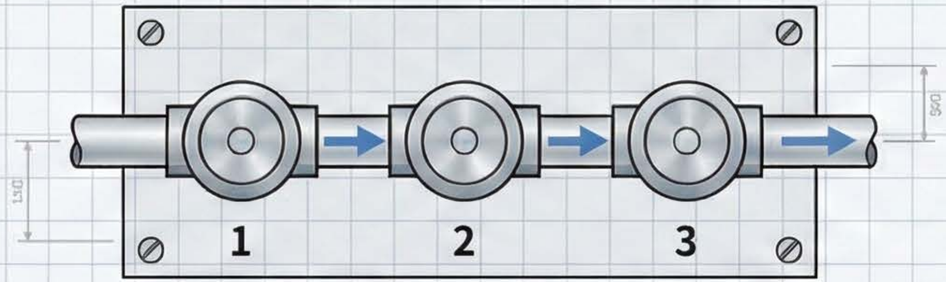
Docstring (說明文件)

回傳值 (Output/Product)

注意：如果沒有 return，函式執行完畢後將回傳 None（就像反應器沒有出口管線）。

參數與引數：靈活的控制介面

位置參數 (Positional Arguments)

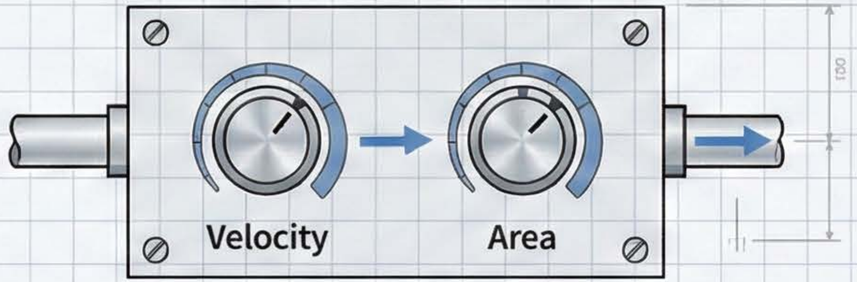


依順序對應，必須精確。就像依序開啟閥門。

```
calc_flow(100, 2.5)
```

風險：順序錯誤會導致嚴重後果 (如將溫度填成壓力)。

關鍵字參數 (Keyword Arguments)



指定名稱賦值，順序不重要。

```
calc_flow(velocity=2.5, area=100)
```

優勢：程式碼可讀性高，不易出錯。

預設參數 (Default Arguments)

概念：設備的「原廠設定值」。若未指定，則使用預設值。

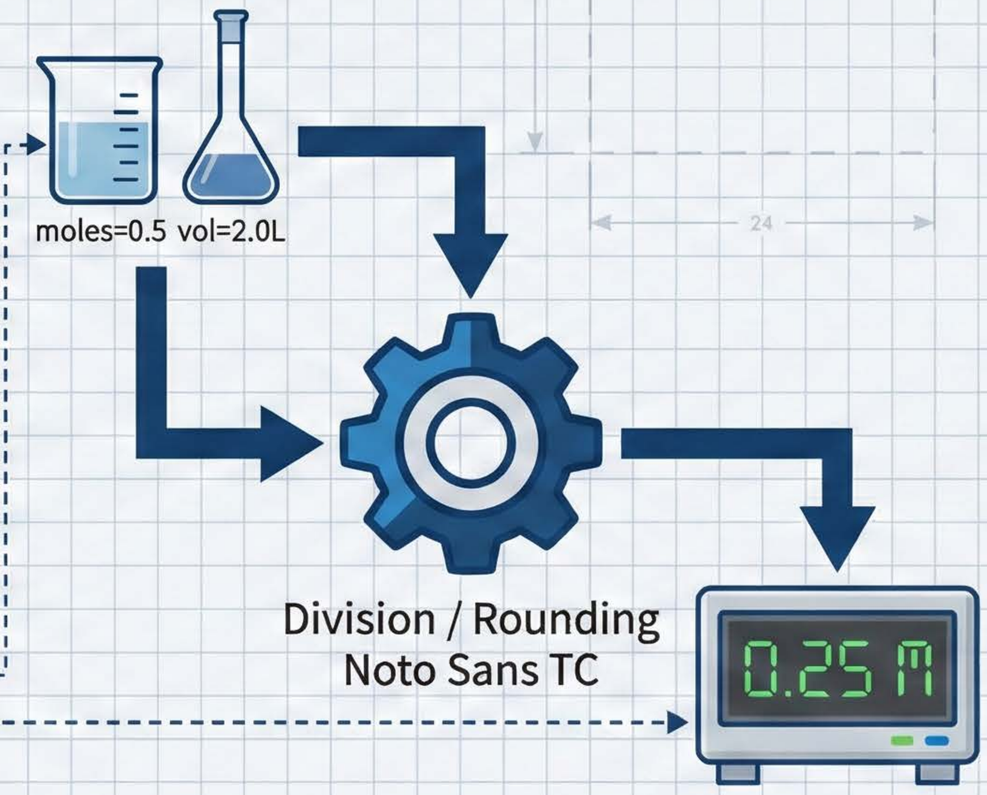
```
def heat_exch(temp, efficiency=0.8):
```

NotebookLM

工程實例：濃度計算函式

建立一個計算摩爾濃度 (Molarity) 的通用工具

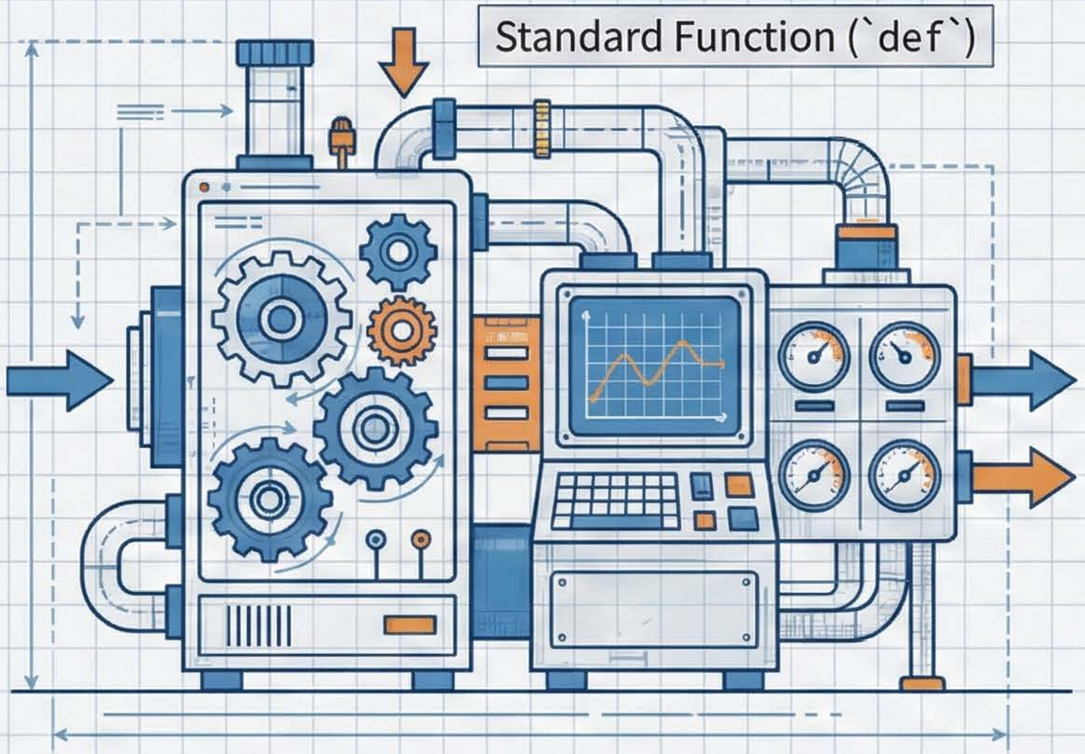
```
def calculate_molarity(moles, volume_liters):
    """
    計算溶液的摩爾濃度 (M)
    """
    if volume_liters <= 0:
        return "Error: Volume must be positive"

    molarity = moles / volume_liters
    return round(molarity, 2)

# 呼叫函式
result = calculate_molarity(moles=0.5, volume_liters=2.0)
print(f"濃度: {result} M")
```

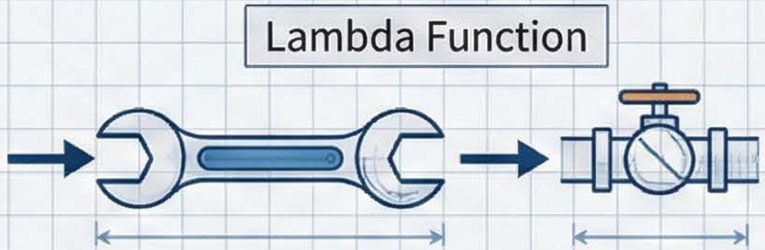


NotebookLM

Lambda 函式：輕量級的管線工具

匿名函式 (Anonymous Function)





輸入

單行運算邏輯

```
lambda arguments : expression
```

Standard

```
def double(x):
    return x * 2
```

Lambda

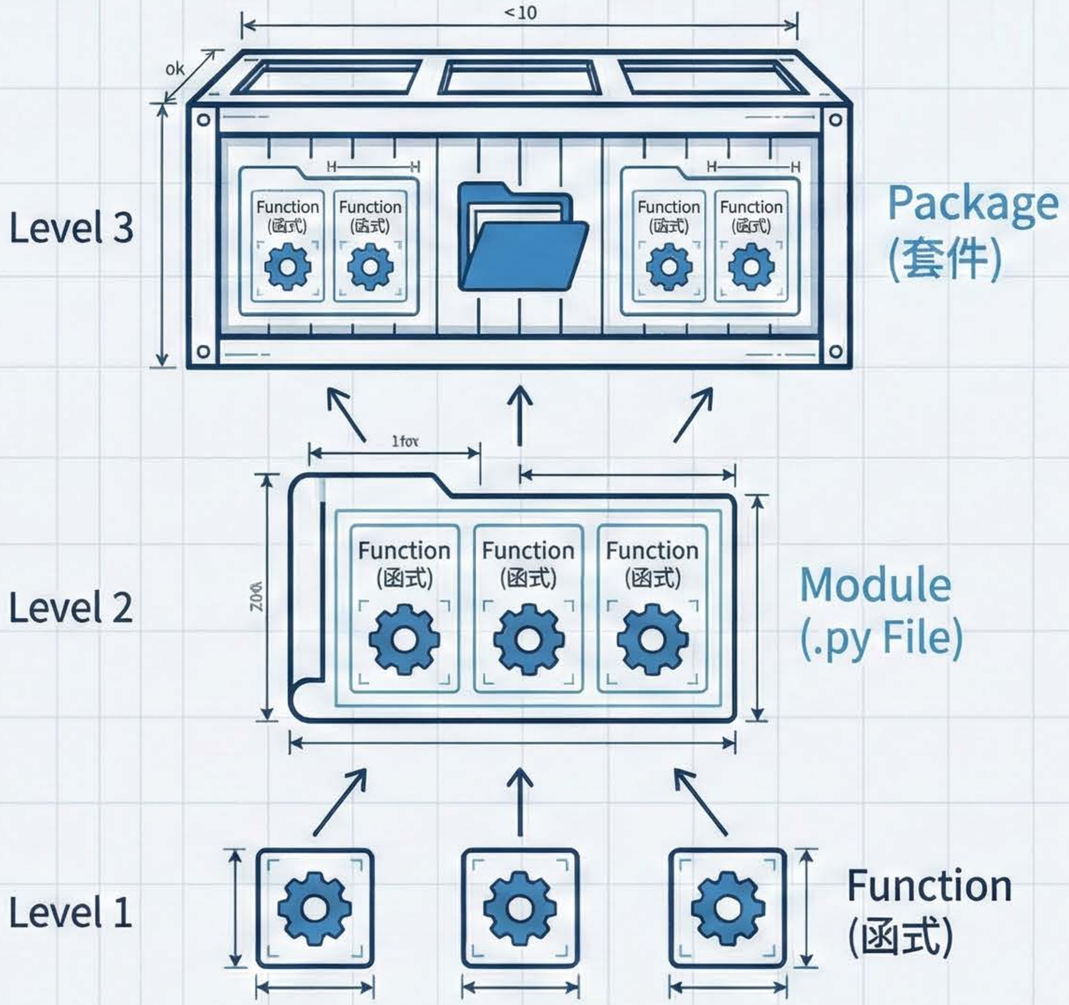
```
double = lambda x: x * 2
```

特性 (Characteristics)

- 不需要名稱 (def)
- 僅限單行程式碼
- 適合用於簡單數學轉換或數據處理管線 (map , filter)

模組 (Modules)：系統化的廠房規劃

從單元操作到系統整合



Why Modules?

- 組織管理 (Organization)：將相關功能的程式碼分類 (例如：`reactor.py`，`thermo.py`)。

- 維護性 (Maintenance)：修改單一檔案即可更新系統，就像維修特定廠區不影響全廠。

- 重用性 (Reusability): 跨專案共享工具箱。

NotebookLM

導入模組：連接系統管線 連接系統管線

使用 import 關鍵字載入外部功能

導入整個模組 (Import Module)

Roboto Mono

```
import math ──────→ math.sqrt(25)
```

搬進整個工具箱，使用時需指名「工具箱裡的錘子」。

導入特定功能 (From ... Import)

```
from math import sqrt ──→ sqrt(25)
```

只把錘子拿出來放在桌上，直接使用。(注意名稱衝突風險)

取別名 (Alias)

```
import pandas as pd ──→ pd.DataFrame()
```

給工具箱取個簡稱(業界標準慣例)。

NotebookLM

實作練習：建立自定義模組

reactor_utils.py
(The Module)

```
# 定義工具函式
def conversion(inlet, outlet):
    return (inlet - outlet) /
    inlet * 100
```

Import

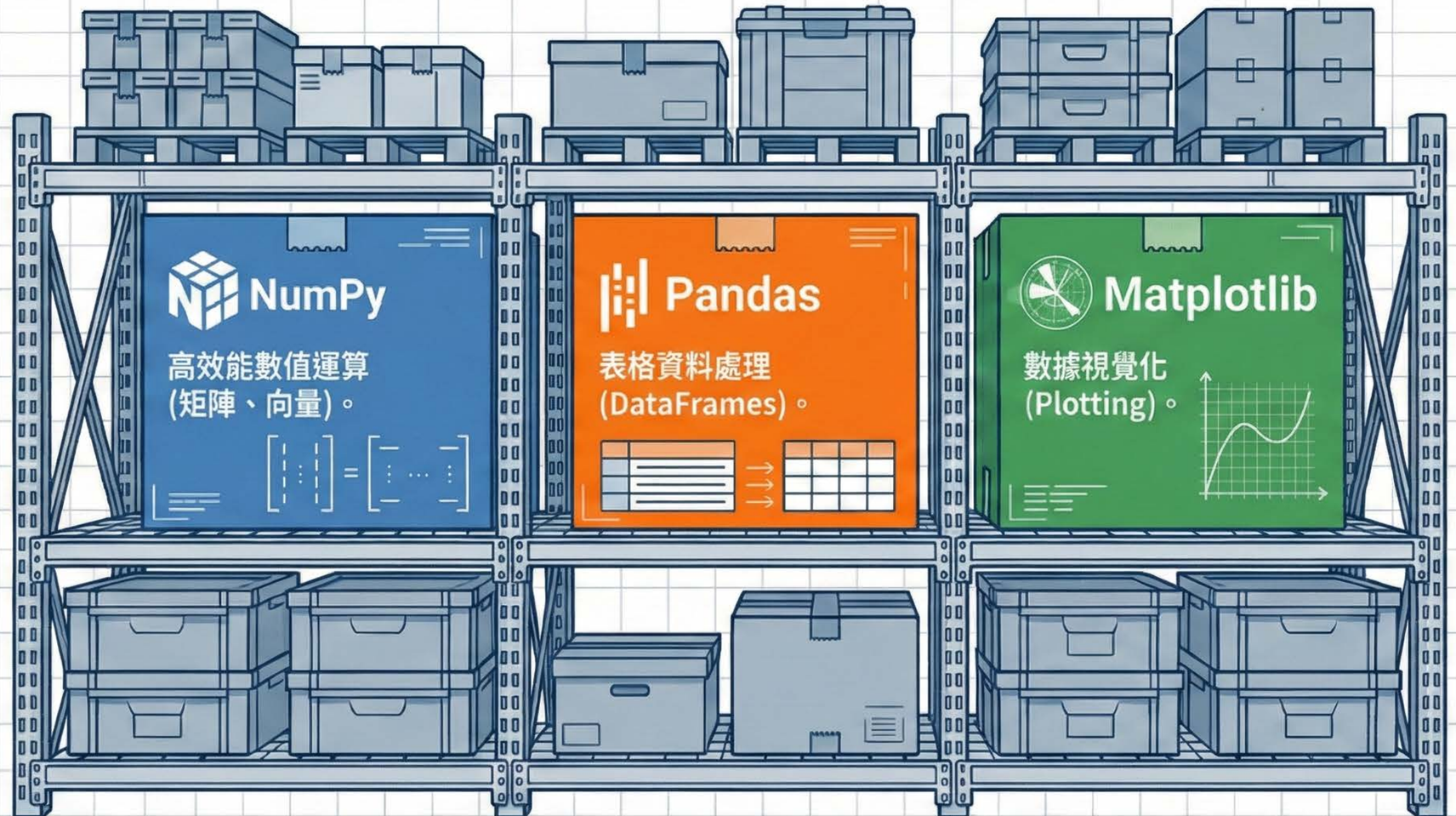
main.py
(The Program)

```
import reactor_utils as ru

# 使用模組內的函式
conv = ru.conversion(100, 20)
print(f"轉化率: {conv}%")
```

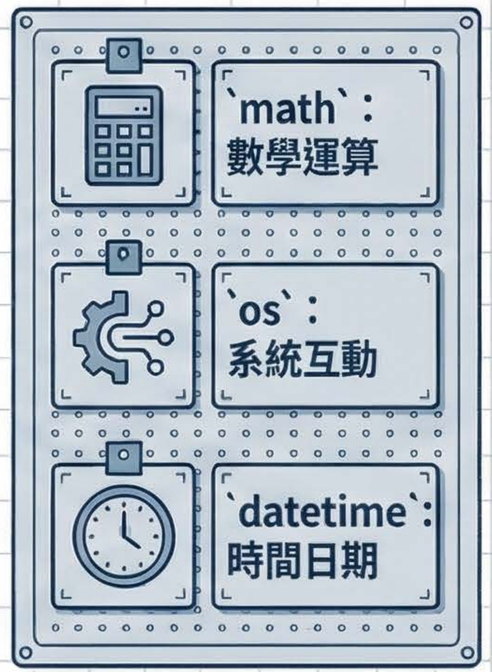
確保兩個檔案位於同一目錄下 (Directory)。

NotebookLM

生態系統：站在巨人的肩膀上

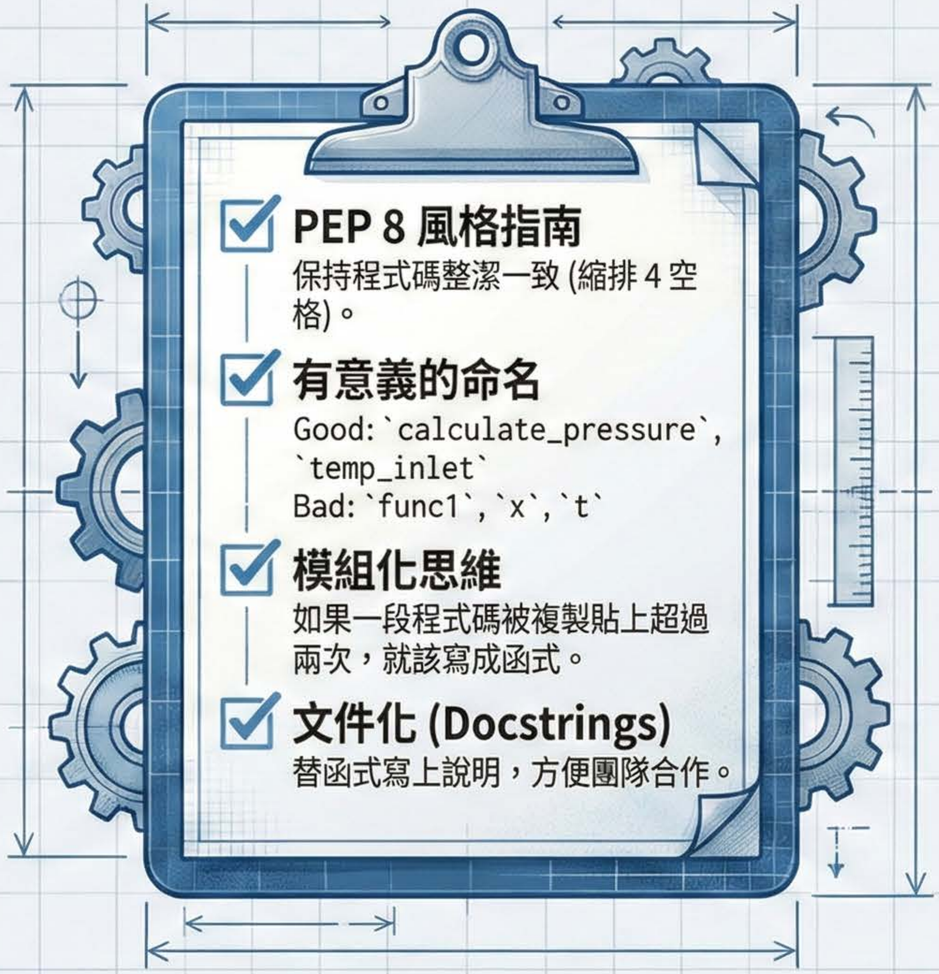


Standard Library
(內建標準庫)



不要重新發明輪子 (Don't Reinvent the Wheel) — 善用現成的高效能套件。

最佳實踐：程式碼的品質管理 (QA)



"Code is read much more often than it is written."

NotebookLM

重點回顧：模組化設計清單

Section 05: 函式 (Functions)

- ✅ 封裝邏輯：輸入 (Args) -> 處理 -> 輸出 (Return)。
- ✅ 使用關鍵字參數提高可讀性。
- ✅ Lambda：簡潔的單行函式。

Section 06: 模組 (Modules)

- ✅ 檔案即模組： 利用 .py 檔案組織程式碼。
- ✅ Import 機制：靈活連結不同工具箱。
- ✅ 生態系統：熟練使用 NumPy 與 Pandas。

下一單元預告：例外處理 (Exception Handling) —— 讓您的工廠具備自動安全連鎖系統。